

BACKSTORY AUTOMATION CATALOG

Zapier implementation guide for the 21-meeting-intelligence-normalizer workflow

Implementation guide · Generated 2026-05-31

Meeting Intelligence Normalizer — Zapier Implementation Guide

This file is a plain-English implementation guide. It is not an importable JSON artifact.

Zapier does not accept a reusable “upload this workflow JSON” format for the kind of cross-workspace assets this catalog is describing. To replicate this workflow in Zapier, build it as one or more Zaps plus, when needed, a custom Zapier app. If you want a reusable public Zap Template, every integration in the template must be a public integration, and Zapier rejects templates that include Code, custom Webhook, or custom Formatter steps. If you are creating workflows through Zapier workflow-creation APIs, use Public Apps because private apps are not supported there.

Workflow Summary

- Workflow ID: 21-meeting-intelligence-normalizer
- Category: platform-enablement
- Source trigger in this catalog: Schedule / Webhook |
- Recommended Zapier trigger surface: Schedule by Zapier
- Delivery targets: Microsoft Teams channel or chat, Calendar event or task

What To Build In Zapier

- Microsoft Teams app actions or companion Microsoft 365 app steps
- Google Calendar app actions for event and task creation
- Meeting-system native app or published custom Zapier app for transcript retrieval
- Zapier Tables or Storage for state, routing maps, and dedupe keys

Recommended Implementation Shape

1. Confirm the required public or workspace-native Zapier apps are available before you build the workflow.

2. Create the primary Zap with the trigger and the minimum deterministic steps needed to gather source data and reach structured synthesis.
3. Split delivery, approvals, or fan-out into additional Zaps when the workflow naturally decomposes or when public-template restrictions would block a single-Zap design.
4. Use Zapier Tables or Storage for routing maps, dedupe keys, approval state, or cross-Zap handoff instead of hiding state in code.
5. If you plan to publish a public Zap Template, test the template path separately from any internal/private workspace build.

Custom App Actions To Provide

- `meeting_source_fetch_artifact`
- `load_routing_map`

Zaps To Create

- `21_meeting_intelligence_normalizer_primary_zap` — Trigger: Schedule by Zapier. Native steps: Native app step: Fetch Source Payloads -> Native app step: Normalize Meeting Schema -> Native app step: Resolve Account Association -> AI step backed by a native LLM app returning strict JSON for AI Transcript QA -> Native delivery step: native app action for Publish Canonical Meeting Event. Notes: Keep the primary Zap focused on normalization, source access, and structured synthesis. Route secondary fan-out through additional native Zaps when template restrictions or retry requirements demand decomposition.
- `21_meeting_intelligence_normalizer_delivery_or_followup_zap` — Trigger: Zapier Tables state change, Interfaces action, or an upstream custom app event. Native steps: Read `delivery_payload` and target metadata from structured fields -> Microsoft Teams app action or Microsoft 365 companion action -> Google Calendar or Microsoft Outlook Calendar action. Notes: Use this secondary Zap when you need reusable delivery fan-out, fallback paths, or approval boundaries without falling back to Webhooks or Code steps.

Contracts To Preserve

- `run_context` : workflow-level execution context such as trigger, mode, lookback, delivery mode, and dry-run state.
- `source_record` : normalized business record from the source adapter or source-system action layer.
- `enrichment_context` : MCP or native app enrichment results used only during synthesis.
- `delivery_payload` : deterministic delivery fields for the final native connector or app action.

Structured AI Requirements

- `ai_transcript_qa` : return JSON only with keys `title` , `summary` , `confidence` , `recommended_actions` , `source_refs` . Intent: AI Agent flags weak transcripts, missing attendees, ambiguous action items, or poor account mapping before the event is reused downstream.

Validation Checklist

- Do not treat this guide as an importable JSON workflow; build the workflow as one or more Zaps and, if needed, a custom Zapier app.
- If you want a reusable public Zap Template, only public integrations can be used in the template.
- Do not include Code, custom Webhook, or custom Formatter steps in a public Zap Template.
- If you are creating workflows through Zapier workflow-creation APIs, use Public Apps only and avoid Paths because they are not currently supported there.
- Validate structured AI output before any Slack, Teams, email, calendar, Jira, or sink action runs.

Official References

- [Zap templates restrictions](#)
- [Known limitations for workflow creation](#)
- [Build with CLI](#)
- [Zapier action design](#)
- [Add a create action](#)
- [AI actions](#)
- [Custom actions and API requests actions](#)